# Measure first: before and after



Two class histograms around a workload show what it allocated.

```
jps -l                                                  # find <pid>
jcmd <pid> GC.class_histogram -all > before.txt
# ... run the workload ...
jcmd <pid> GC.class_histogram -all > after.txt
```

- `-all` also counts unreachable objects, so garbage shows up, and no full GC runs first.
- Subtract per class: that is how `prof.HistoProfiler` counts objects per call.

# JOL: the layout of one object



Java Object Layout (JOL) reports how the JVM lays an object out: header, fields, padding and size.

```
import org.openjdk.jol.info.ClassLayout;
import org.openjdk.jol.info.GraphLayout;

System.out.println(ClassLayout.parseClass(String.class).toPrintable());

long bytes = GraphLayout.parseInstance(list).totalSize();   // the object and all it references
```

- `ClassLayout` shows one object; `GraphLayout` adds up everything reachable from it.
- JOL is an OpenJDK project, added as a library; it is not part of the JDK.

# JOL with compact object headers



| HashMap$Node | Header | Fields | Size |
|---|---|---|---|
| Default | 12 B | hash, key, value, next: 16 B | 32 B (4 B padding) |
| `-XX:+UseCompactObjectHeaders` | 8 B | the same 16 B | 24 B |

- Run the same JOL code with the flag off and on, and compare the sizes.
- The 4 B saved only counts when the total drops below the next 8 B boundary.

# DEMO Demo: one map entry, headers off and on



```
Class<?> node = Class.forName("java.util.HashMap$Node");
System.out.println(ClassLayout.parseClass(node).toPrintable());
```

```
java -cp jol-core-0.17.jar -XX:-UseCompactObjectHeaders NodeLayout.java   # 32 bytes
java -cp jol-core-0.17.jar -XX:+UseCompactObjectHeaders NodeLayout.java   # 24 bytes
```

- Same fields, 4 B less header, and the 4 B alignment gap is gone.

# JMH: bytes per call, not only speed



```java
@Benchmark
public boolean jdkAnyMatch()
{
    return jdkList.stream().anyMatch(Integer.valueOf(this.test)::equals);
}

@Benchmark
public boolean ecAnySatisfyWith()
{
    return Iterate.anySatisfyWith(jdkList, Integer::equals, this.test);
}
```

- `-prof gc` reports `gc.alloc.rate.norm`: bytes allocated per call.
- `-prof prof.HistoProfiler` counts the objects per call.

# What one `anyMatch` call leaves behind



Escape analysis off: every object each version creates, counted.

